

Residual Denoising Enables Sample-Efficient Multi-Agent Coordination on Demand

Author Names Omitted for Anonymous Review

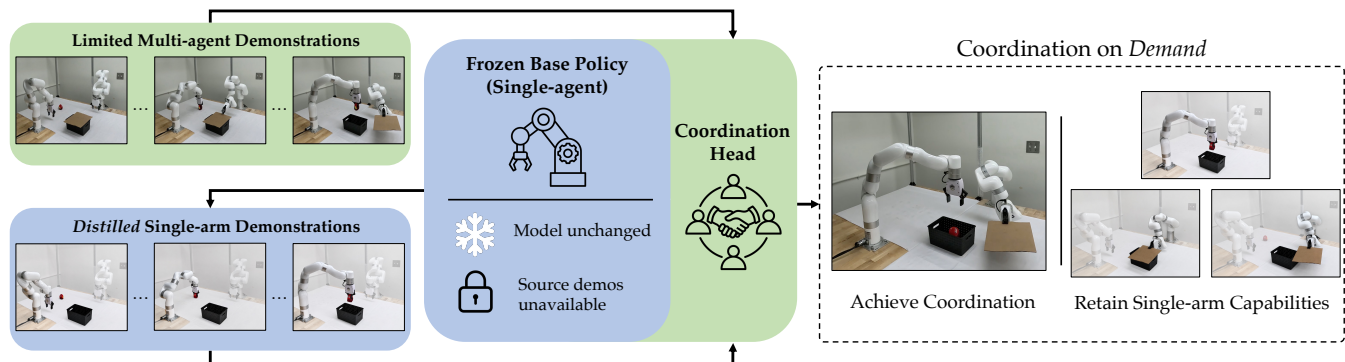


Fig. 1: **Multi-agent adaptation with ALTER.** ALTER adapts a frozen single-arm visual diffusion policy to coordinated multi-agent tasks without access to the original expert demonstrations. Blue denotes the frozen source policy and its policy-distilled rollouts, while green denotes the multi-agent demonstrations and trainable coordination head. Each robot runs the adapted policy using only its own visual observations, enabling decentralized coordination without explicit inter-agent communication while retaining the original single-arm skills.

Abstract—Pretrained robot policies offer strong manipulation skills but are typically limited to single-agent settings, where a robot acts in isolation. In this work, we study how to adapt pretrained single-agent diffusion policies to multi-agent settings using minimal collaborative data, co-optimizing for two key objectives: high coordination performance and single-agent skill retention. To this end, we introduce ALTER, an adaptation method for *coordination on demand*: the adapted policy coordinates with other robots when deployed in a team while remaining capable of acting independently when operating alone. Execution is decentralized: each robot acts only on its own visual observations, without explicit inter-agent communication. Our method trains a *coordination head* that predicts a *residual denoiser* to transform single-agent behavior into coordinated multi-agent behavior when necessary while also preserving single-agent capabilities. To preserve single-agent capabilities, we augment a small number of collaborative demonstrations with self-distilled data generated by the base policy during training of the residual denoiser. In simulation, ALTER achieves higher coordination success over our baselines while retaining much higher source-skill retention. In our hardware experiments, we find similar trends where ALTER better co-optimizes for coordination success and single-agent skill retention than the baselines.

I. INTRODUCTION

Imagine you have just enjoyed a dinner party with friends and now face the less delightful task of clearing the table. Luckily, you have two state-of-the-art robots at home to help.¹ To be useful in this setting (beyond dancing [1] or performing roundhouse kicks [2] in a corner), your robots must *manipulate* their environment: *change* the world around them by picking up objects, opening cabinets, putting objects away, and wiping the table.

¹You could, of course, have asked your friends to help clean up. Why you did not is left to the reader’s imagination.

In principle, generative policies based on flow matching or diffusion models enable robots to learn such skills from demonstrations [3]. In fact, numerous recent works have shown great success in training individual agents to perform complex manipulation skills using this approach [4], [5], [6], but these results remain largely confined to the single-agent domain. Enabling *multiple* robots to collaborate in a *decentralized* fashion could offer a cost-effective way to scale their productivity.² However, extending imitation learning (IL) to this multi-agent setting is challenging because collecting multi-agent data at scale is costly.

In view of the multi-agent data bottleneck, and the fact that there exist strong off-the-shelf policies for single-agent settings, rather than learning coordination behavior from scratch, in this work we treat multi-agent learning as an adaptation problem, starting from a pretrained image-conditioned single-agent diffusion policy, which we call the *base policy* from now on. Starting from such a base policy, we ask

How can we adapt a single-agent diffusion policy to coordinate with others using limited multi-agent data while retaining its single-agent capabilities?

To make our contribution applicable to off-the-shelf policies, we study this question without assuming access to the base policy’s training data.

Our main contribution, *ALTER* (Adaptation from Limited demonstrations for Team coordination with Existing-skill Retention), is a multi-agent adaptation paradigm that tackles precisely this challenge. At a high level, our approach comprises three stages (cf. Fig. 1): (i) we roll out the base

²Much as sharing the task with your friends would have helped, had you asked.

policy to generate demonstrations of single-agent skills (in Fig. 1: picking and placing objects); (ii) the user provides a limited number of multi-agent demonstrations of coordinated behavior (in Fig. 1: opening the box with one robot for another robot to place an object inside); (iii) to achieve multi-agent coordination while retaining single-agent skill during adaptation, we combine the self-distillation data from step 1 with the multi-agent data from step 2 to train a *residual coordination head* that bridges the gap between the frozen single-agent policy and the target coordinated behavior. At deployment, each robot runs its own copy of the adapted policy using only its local visual observations, enabling decentralized coordination without explicit inter-agent communication.

We compare our method to several imitation learning baselines across a suite of collaborative manipulation tasks both in simulation and in hardware. Our results show that ALTER adapts base policies more effectively than the baseline approaches: it achieves higher coordination success while ensuring better retention of single-agent skills (i.e., the ability to act independently even after adaptation).

II. RELATED WORK

We position our work relative to diffusion visuomotor policies, multi-agent imitation learning, and pretrained-policy adaptation.

Diffusion Models provide a flexible policy class for modeling multimodal action distributions [7], [8], [3]. Diffusion Policy [3] generates action chunks from visual observations via conditional denoising. DP3 [9] extends this formulation by conditioning on sparse 3D point clouds. These methods typically train a policy directly for the target task and do not directly support adaptation to new settings without retraining.

Multi-Agent Imitation Learning aims to coordinate behavior directly from collaborative demonstrations. MIMIC-D [10] jointly trains decentralized diffusion policies with a shared loss. CHORUS [11] adapts a shared vision-language-action policy for decentralized multi-embodiment teams. Such methods are typically data-hungry and train directly for collaboration rather than preserving pretrained single-agent capabilities. Other diffusion approaches add explicit teammate models or consensus latent representations to support coordination [12], [13], which require additional teammate-modeling or latent-consensus machinery. CoDi [14], on the other hand, coordinates pretrained single-agent diffusion policies without collaborative demonstrations, but instead applies a user-specified multi-agent cost at sampling time for guidance. However, it can be difficult to specify such a cost function for complex multi-agent coordination.

Adaptation without Forgetting. Our problem can be viewed as a variant of continual learning [15], [16] but with distinct constraints: we do not have the source data and we have highly skewed data budgets in different domains (single- vs multi-agent data). A second approach to adaptation without forgetting is runtime policy selection. Mixture-of-experts models and language-guided skill planners select an expert or temporally extended skill at runtime [17], [18]. In our

deployment setting, however, no external signal indicates whether the robot should execute a source task or collaborative policy.

Positioning of Our Approach. Residual policy learning [19] provides a mechanism for adapting policies by combining a fixed nominal controller with a learned corrective policy. We apply the principle of residual learning at the *denoiser* level. Our architecture is also related to parameter-efficient adaptation methods like side-tuning [20], which adapts a fixed pretrained network through an additive trainable side network, and ControlNet [21], which augments a frozen diffusion model with zero-initialized trainable branches for additional conditioning. Using this idea, we keep a pretrained single-arm diffusion policy frozen and train only a residual head to extend it to the multi-agent coordination target task while preserving the single-arm source task behavior.

III. PRELIMINARIES: DIFFUSION MODELS

Our method builds on conditional diffusion models, which we briefly review in the continuous-noise-level parameterization of Karras et al. [22], commonly referred to as EDM. We refer the reader to [7], [23], [22] for a complete overview.

Diffusion models aim to generate samples from a distribution $p(x | c)$ over data x conditioned on context c . Importantly, the underlying data distribution is unknown. Instead, the generative model must be learned from only a dataset of examples $x \sim p(\cdot | c)$. Diffusion models achieve this by considering a family of noise-corrupted distributions $p_\sigma(x_\sigma | c) := \int p(x | c) \mathcal{N}(x_\sigma; x, \sigma^2 I) dx$ obtained by perturbing data $x \sim p(\cdot | c)$ with Gaussian noise of standard deviation $\sigma \geq 0$. At $\sigma = 0$, this family coincides with the data distribution; at a sufficiently large noise level $\sigma_{\max} \gg 0$, it is indistinguishable from the Gaussian $\mathcal{N}(0, \sigma_{\max}^2 I)$, which is trivial to sample from.

The key idea of diffusion models is to generate data samples from Gaussian noise by reversing the noise-corruption process. Starting from a sample $x_{\sigma_{\max}} \sim \mathcal{N}(0, \sigma_{\max}^2 I)$, we denoise it into a clean sample by integrating the following probability flow ordinary differential equation (ODE) [23]

$$\frac{dx_\sigma}{d\sigma} = -\sigma s(x_\sigma, \sigma, c) \quad (1)$$

from $\sigma = \sigma_{\max}$ down to $\sigma = 0$, where $s(x_\sigma, \sigma, c) := \nabla_{x_\sigma} \log p_\sigma(x_\sigma | c)$ is the score function of the noised distribution. Intuitively, the score function points towards high-probability regions of the underlying data distribution $p(x | c)$.

Since the score function s_σ in (1) is unknown, diffusion models learn it from data. In the EDM formulation, this is achieved by training a denoiser $D_\theta(x_\sigma, \sigma, c)$ with parameters θ , a network that predicts the clean x from its noised version, by denoising regression using the following loss function

$$\mathcal{L}_{\text{den}}(\theta; \mathcal{D}) = \mathbb{E}_{\mathcal{D}, \sigma, \epsilon} \left[w(\sigma) \| D_\theta(x + \sigma\epsilon, \sigma, c) - x \|_2^2 \right], \quad (2)$$

where the expectation is over $(x, c) \sim \mathcal{D}$, $\sigma \sim p(\sigma)$, and $\epsilon \sim \mathcal{N}(0, I)$, and the noise-level distribution $p(\sigma)$ and the

weighting $w(\sigma) > 0$ are design choices of the training recipe [22]. By Tweedie’s formula [24], [22], the minimizer D^* of (2) is the posterior mean $\mathbb{E}[x | x_\sigma, c]$ and satisfies

$$D^*(x_\sigma, \sigma, c) = x_\sigma + \sigma^2 s(x_\sigma, \sigma, c). \quad (3)$$

Therefore, any learned denoiser D_θ induces a score estimate $s_{D_\theta}(x_\sigma, \sigma, c) := (D_\theta(x_\sigma, \sigma, c) - x_\sigma) / \sigma^2$ that we can use to estimate data samples via (1). In this work, we solve the ODE (1) numerically using Algorithm 1 in [22].

IV. PROBLEM FORMULATION

In this section, we formalize the problem of adapting a pretrained single-arm policy for multi-arm coordination.

Task Model. We consider a collaborative task between N homogeneous agents over a finite horizon of T steps. Throughout this work, we use the superscript i and the subscript t to denote a quantity associated with agent i at time t . The agents operate in a shared workspace and perceive it only through their own sensors. Hence, we model the collaborative task as a Dec-POMDP, $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, P, Z, \rho_0, N \rangle$ [25]. Here, $\mathcal{S}$ is the global state space of the workspace, which includes all agents and objects; $\mathcal{A} = \mathcal{A}^1 \times \dots \times \mathcal{A}^N$ is the joint action space, where $\mathcal{A}^i = \mathbb{R}^{d_a}$ is the action space of agent i and d_a is the action dimension of each agent; $\mathcal{O} = \mathcal{O}^1 \times \dots \times \mathcal{O}^N$ is the joint observation space, where $\mathcal{O}^i \subseteq \mathbb{R}^{d_o}$ is the continuous local observation space of agent i and d_o is the per-agent observation dimension; P is a Markov transition kernel where for every global state $s \in \mathcal{S}$ and joint action $a_t^{1:N} := (a_t^1, \dots, a_t^N) \in \mathcal{A}$, $P(\cdot | s, a_t^{1:N})$ is a probability distribution over the next global state in $\mathcal{S}$; $Z = (Z^1, \dots, Z^N)$ collects the per-agent observation kernels $Z^i(\cdot | s)$ which are probability distributions over the local observation space $\mathcal{O}^i$ given global state s ; and ρ_0 is the initial-state distribution.

Decentralized Execution. Centralized planning and explicit inter-agent communication can be unreliable or unavailable at deployment. Therefore, we require each agent to make decisions based only on its own local observation while coordinating with other agents sharing the same workspace. Assuming that all agents are driven by the same policy, i.e., $\pi^i = \pi$, we model the joint policy under this decentralized execution assumption as

$$\pi^{1:N}(a^{1:N} | o^{1:N}) := \prod_{i=1}^N \pi^i(a^i | o^i) = \prod_{i=1}^N \pi(a^i | o^i). \quad (4)$$

Note that sharing policy parameters does not weaken decentralization because each agent receives only its own observations. In summary, starting from initial states s_0 drawn from ρ_0 , at every time step, each agent receives an observation and selects an action, and the joint action of all agents drives the workspace to its next state according to the following closed-loop stochastic dynamics

$$o_t^i \sim Z^i(\cdot | s_t), a_t^i \sim \pi(\cdot | o_t^i), s_{t+1} \sim P(\cdot | s_t, a_t^{1:N}). \quad (5)$$

Diffusion Policies with Action Chunking. We represent the policy π as a conditional diffusion model as introduced in

Sec. III, where the distribution now is over an agent’s *actions* (the *data* x in §III) conditioned on local RGB images of the scene (the *context* c in §III). As is common practice [3], we instantiate diffusion policies with action chunking [3], [26]: at time t , agent i predicts an H -step action chunk $a_{t:t+H}^i := (a_t^i, \dots, a_{t+H-1}^i) \in \mathbb{R}^{H \times d_a}$ from its observation o_t^i , where H is the prediction horizon, executes the first $H_a \leq H$ actions, and then replans from its next observation. With a slight abuse of notation, we therefore let $\pi(\cdot | o_t^i)$ denote a distribution over action chunks $a_{t:t+H}^i$, and the dynamics (5) execute chunks in this receding-horizon fashion.

Collaborative Demonstrations. We denote by $\mathcal{D}_C$ a set of M_C expert episodes in which all N agents perform a collaborative task together,

$$\mathcal{D}_C = \{\tau_m^C\}_{m=1}^{M_C}, \quad \tau_m^C = \{(o_t^{1:N}, a_t^{1:N})\}_{t=0}^{T-1}, \quad (6)$$

where each episode τ_m^C records the joint observations $o_t^{1:N}$ and joint actions $a_t^{1:N}$ at every time step.

The Adaptation Problem: Coordination on Demand. The goal of decentralized multi-agent imitation learning is to learn a homogeneous policy π such that when each agent employs a copy of π in a decentralized fashion, the joint behavior under (5) reproduces the coordinated behavior in $\mathcal{D}_C$. Rather than learning coordinated behavior from scratch based on only $\mathcal{D}_C$, in this work, we assume access to a *pretrained policy* π_{base} (represented as a diffusion model with a corresponding denoiser D_{base}) that we will use as a starting point for adaptation. Going forward, we shall refer to the single-agent tasks that this base policy was trained to perform as *source tasks* and call the target multi-agent tasks in $\mathcal{D}_C$ the *collaborative tasks*.

We consider deployment conditions in which a robot may operate alone in some episodes and alongside partners in others and assume that each agent receives no label indicating whether collaboration is required (i.e., whether the current episode features one or multiple agents). Therefore, we seek to train an adapted policy that can seamlessly transition between single-agent skills and multi-agent coordination on demand. Hence, we must adapt π_{base} to the collaborative (multi-agent) setting while preserving its source-task (single-agent) capabilities.

Challenges of Multi-Agent Adaptation. The adaptation of a pretrained single-agent policy to a collaborative multi-agent setting is complicated by two main challenges. First, a key challenge is that the collaborative demonstrations $\mathcal{D}_C$ are limited in number since collecting multi-agent demonstrations at scale is difficult in practice due to the need to operate multiple robots simultaneously. Therefore, we aim to learn coordinated behavior from a small number of multi-agent demonstrations, i.e., in the low-data regime.

Second, while we have access to a pretrained policy π_{base} , we may not have access to the original expert demonstrations that were used to train it. This assumption reflects a common deployment scenario in which robot policies are distributed as checkpoints without their training demonstrations [4], [5], [6]. Without the original expert data, we cannot rehearse the

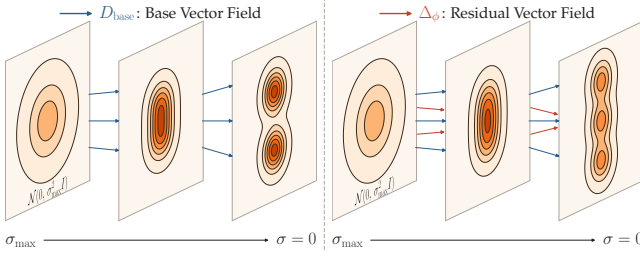


Fig. 2: **The residual as a correction to the base vector field.** *Left:* the frozen base field of D_{base} (blue) alone yields the action distribution of π_{base} . *Right:* adding the residual field Δ_ϕ (red) at every noise level bends the same trajectories toward the coordinated action distribution.

source dataset alongside new collaborative data, which is the standard defense against catastrophic forgetting [27]. Any supervision about source-task behavior must therefore come from π_{base} itself.

V. ALTER

We now present our main contribution, ALTER (Adaptation from Limited demonstrations for Team coordination with Existing-skill Retention), to tackle the adaptation problem outlined in Sec. IV. Fig. 1 illustrates the high-level approach of our method, which rests on three key decisions:

- i. First, we keep the pretrained denoiser D_{base} (parameterizing π_{base}) frozen and add a small residual coordination adapter on top of it so that we can maximize the use of skills already learned by D_{base} while learning coordination (§V-A).
- ii. Second, we replace the missing source dataset with *policy-distilled replay*, a dataset of rollouts of the frozen policy in its source environment (§V-B).
- iii. Third, we train the adapter on the collaborative demonstrations and the replay data jointly, so that a single set of parameters serves both source and target tasks at deployment (§V-B).

Below, we provide more details on each one of these components.

A. Residual Coordination Adapter

The pretrained denoiser already provides competent per-arm behavior; what it lacks is the partner-dependent adjustment that turns individually competent robots into a coordinated team. We therefore leave D_{base} untouched and train a *residual coordination adapter* Δ_ϕ , a second small denoising network with trainable parameters ϕ , whose output corrects the prediction of the frozen base. The adapter receives the same noised chunk, noise level, and observation as the base denoiser, and additionally the base prediction itself since the quantity it must produce is precisely a correction to the base prediction. Crucially, as the residual adapter receives only agent i 's local observation o^i , the same input as D_{base} , decentralized execution as in (5) is preserved by construction.

Thus, the adapted denoiser sums the two outputs,

$$D_\phi(x_\sigma, \sigma, o) = D_{\text{base}}(x_\sigma, \sigma, o) + \Delta_\phi(x_\sigma, \sigma, o, D_{\text{base}}(x_\sigma, \sigma, o)). \quad (7)$$

The adapted policy π_ϕ is a diffusion policy that samples action chunks through ODE (1) with D_ϕ as its denoiser. To simplify the notation, in the remainder of the paper, we abbreviate the residual by $\Delta_\phi(x_\sigma, \sigma, o)$, leaving its dependence on the base prediction implicit.

Remark 1 (Additive Property of Residual Denoisers): Since $D \mapsto (D - x_\sigma)/\sigma^2$ is affine, writing $D_\phi = D_{\text{base}} + \Delta_\phi$ as in (7) gives

$$s_{D_\phi}(x_\sigma, \sigma, o) = s_{D_{\text{base}}}(x_\sigma, \sigma, o) + \sigma^{-2} \Delta_\phi(x_\sigma, \sigma, o), \quad (8)$$

so the probability-flow ODE (1) under s_{D_ϕ} is that of D_{base} with the additional drift $-\Delta_\phi(x_\sigma, \sigma, o)/\sigma$. In particular, whenever $\Delta_\phi(\cdot, \cdot, o) \equiv 0$ for a given o , the sampler evaluates $D_\phi(\cdot, \cdot, o) = D_{\text{base}}(\cdot, \cdot, o)$, so $\pi_\phi(\cdot | o) = \pi_{\text{base}}(\cdot | o)$.

We zero-initialize the output pathway of the adapter, following the convention for residual adapter modules [21], so that $\Delta_\phi \equiv 0$ at initialization. By Remark 1, training therefore starts from a policy that is exactly π_{base} .

Properties of Coordination via a Residual Denoiser. By Remark 1, the adapter is an additive correction to the score field of the base policy at every noise level, and D_ϕ is plugged into the sampler of (1) with no change to the noise schedule or the solver. Fig. 2 illustrates this view. Because the correction acts on the vector field rather than on a sampled action, π_ϕ can modify the distribution of π_{base} , unlike residual policy learning that offsets the output of a base controller [28], [19]. As shown in Sec. VI, this residual adaptation of D_{base} improves data efficiency since the frozen prior π_{base} already carries the basic skills that the collaborative task reuses. Hence, the adapter needs to represent only the inter-agent couplings that the collaborative task requires.

B. Mixed-Domain Training

The residual adapter gives π_ϕ room to learn coordination. However, when training this adapter, we must ensure that we still preserve the source-task behavior. The residual formulation (7) allows us to implicitly capture this: the adapter Δ_ϕ needs to learn to output zero in single-agent settings when no coordination is necessary while providing necessary non-zero corrections for collaborative tasks. In Fig. 2, this means leaving the base field untouched on source-task observations (left) and correcting it only when a partner is in view (right). To achieve this, we train ϕ on both single-agent (source) and multi-agent (coordination) demonstrations simultaneously using a combination of two loss terms which we discuss next.

Policy-Distilled Replay. Recall that, as per our problem formulation (§IV), we do not have access to the source task training dataset. Hence, before we can present our training objectives, we must first discuss how to generate suitable single-agent data for the adapter training. To this

end, we execute the frozen π_{base} in the source environment, following a receding-horizon loop, and record the resulting M_S episodes of observations and executed actions, $\mathcal{D}_S = \{\tau_j^S\}_{j=1}^{M_S}$ with $\tau_j^S = \{(o_t, a_t)\}_{t=0}^{T-1}$. We call $\mathcal{D}_S$ *policy-distilled replay*: it plays the role of rehearsal in continual learning [27], [29] with no expert effort.

Coordination Loss. On the collaborative demonstrations, we apply the denoising loss of (2) to the adapted denoiser, $\mathcal{L}_{\text{coord}}(\phi) = \mathcal{L}_{\text{den}}(\phi; \mathcal{D}_C)$, with the same noise distribution $p(\sigma)$ and weighting $w(\sigma)$ as in the base policy’s training and with gradients flowing only into ϕ . Because all agents share the policy, we pool the per-agent action-observation-chunk tuples $(o_t^i, a_{t:t+H}^i)$ of all N agents in $\mathcal{D}_C$ during training, so no training sample contains another agent’s observation.

Replay Loss. On the replay data, we penalize the residual itself rather than regressing D_ϕ onto the replayed actions, because by Remark 1, a zero residual recovers π_{base} exactly. Therefore, it suffices to drive the residual to zero on source-task inputs, and we penalize it directly,

$$\mathcal{L}_{\text{replay}}(\phi) = \mathbb{E}_{\mathcal{D}_S, \sigma, \epsilon} \left[w(\sigma) \|\Delta_\phi(x + \sigma\epsilon, \sigma, o)\|_2^2 \right], \quad (9)$$

where the expectation runs over $(x, o) \sim \mathcal{D}_S$ with the same σ and ϵ distributions as in (2).

Combined Adaptation Objective. We train the adapter on the weighted sum of the two objectives, $\mathcal{L}(\phi) = \mathcal{L}_{\text{coord}}(\phi) + \lambda_{\text{ret}} \mathcal{L}_{\text{replay}}(\phi)$, where the retention weight $\lambda_{\text{ret}} > 0$ trades coordination gain against source-task retention. This objective drives the residual to zero when the observation resembles the source task and makes it corrective when it shows a partner at work. As a result, we do not require an explicit routing mechanism inside the adapter to switch between single-agent and multi-agent behavior.

C. Deployment and Implementation Details

Since all agents share one set of weights, we instantiate a single copy of the frozen D_{base} together with Δ_ϕ and deploy it on every agent, each acting only on the RGB images from its own cameras. Every agent runs the pretrained policy’s receding-horizon loop with D_ϕ as its denoiser and receives no signal about whether a partner is present (§IV). Execution is fully decentralized: agents exchange no information and coordinate only through the physical workspace, as modeled in (5). Because the adapter is smaller than the backbone, it adds little inference cost over the pretrained policy.

Our base denoiser is a DiT-style transformer [30] that consumes the noised chunk, the noise level, and visual tokens produced by an image encoder; ALTER touches this network only through its forward pass. We implement Δ_ϕ as a small DiT-style decoder with the EDM preconditioning of [22]. It is conditioned on two things. First, it uses the base encoder’s token features through learned projections, in the spirit of side-tuning [20]. Second, the raw observations are also processed through a lightweight convolutional network. This lets the adapter attend to partner cues that the frozen encoder, trained without partners in view, may discard. Figure 1 (middle) depicts both networks.

VI. SIMULATION EXPERIMENTS

We conduct two-arm simulation experiments to answer three main questions. (Q1) Does ALTER achieve higher coordination success than training from scratch when multi-agent demonstrations are limited? (Q2) Does ALTER preserve source-task behavior? (Q3) How does the coordination-head size of ALTER affect coordination and source success when multi-agent demonstrations are limited?

A. Experiment Settings

Two-Arm Task. We use `TwoArmPlaceWipe` as our main benchmark, where two robots coordinate from local visual observations while manipulating objects in a shared workspace. In the intended task sequence, Robot 1 lifts a handled tray, places it on one of two waiting areas, and returns it. Robot 2 picks up a sponge to wipe up the dirt under the tray and returns the sponge to the sponge pad. This task requires spatial and temporal coordination between the two arms in a shared space: Robot 1 must clear the dirt grid before Robot 2 can wipe it and must delay returning the tray until Robot 2 has completed the wipe. Any break in this sequence would likely lead to robot collision and task failure. The underlying frozen single-arm policy provides the place-and-return and wipe-and-return source behaviors. We randomize the initial positions of objects, pads, and dirt across demonstrations and evaluation episodes to provide diverse task instances and evaluate generalization to configurations absent from the demonstrations.

Training Protocol. We train using AdamW with a constant learning rate of 2×10^{-4} and weight decay of 1×10^{-4} for 900k optimizer steps, saving exponential-moving-average-filtered checkpoints every 100k steps.

Evaluation Protocol. We evaluate each selected policy in both the multi-agent and single-arm domains. We report *coordination success*, the fraction of multi-agent episodes satisfying the task-specific success criteria (described below), and *source success*, the success rate on each original single-arm task. We select the checkpoint with the highest coordination success over 40 evaluation episodes, then evaluate it on 200 fresh multi-agent episodes. To assess source-task retention, we evaluate the same checkpoint on fresh episodes of the original single-arm tasks, keeping the coordination head active for our method, and compare source success with the unadapted base policy. We resample each model every 15 environment steps. We use a 1,800-step episode cap as a hard cutoff for these samples. Table I reports the completed multi-agent evaluations.

Coordination Success Criteria. This task uses coordination success as the main evaluation metric. We count an episode as successful when the following conditions hold: the tray is less than 1.5 cm from the return position along both the x and y axes; the sponge is offset no more than 8 cm along x and 6 cm along y from its target; and wipe coverage reaches at least 60%.

Training Data and Budgets. Since collecting coordinated demonstrations requires operating multiple robots simultane-

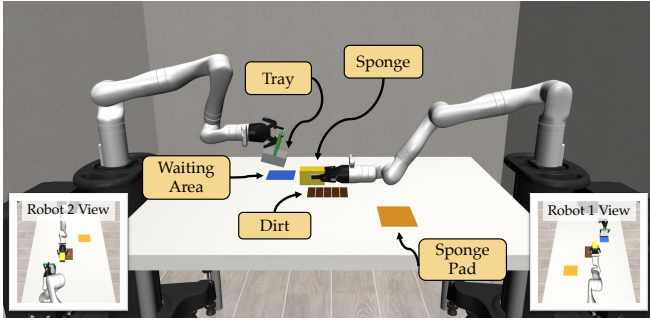


Fig. 3: In the TwoArmPlaceWipe simulated task, one robot places and returns a tray while another wipes up dirt.

ously, it is desirable to achieve policy adaptation with as few multi-agent demos as possible. Therefore, we study policy performance under limited multi-agent data budgets. For the TwoArmPlaceWipe task, mixed-data training includes 20, 40, and 60 single-arm demonstrations, respectively. These single-arm demonstrations are policy-distilled rollouts from the pretrained policy and are entirely separate from the 600 expert single-arm demonstrations used to train the base policy. Multi-agent-only training reuses the same multi-agent demonstrations.

For mixed-data training, each optimizer update processes a batch of 512 samples, split equally between multi-agent and single-arm data. Multi-agent-only training uses 256 multi-agent samples per update.

B. Baselines

Our evaluation compares ALTER with three baselines to examine how training from scratch or fine-tuning the pretrained policy affects coordination success and source-task retention under limited multi-agent data. We condition all methods only on RGB image observations from their own shoulder camera. Hence, all approaches are *decentralized* at test time.

From Scratch (FS). To test the benefit of reusing pretrained single-arm behavior, we train a baseline policy from scratch using the same multi-agent demonstrations and policy-distilled single-arm rollouts that we use to train the coordination head. Its parameter count closely matches the sum of the coordination-head and frozen single-arm-policy parameter counts; we detail the matching procedure in Section VI-E.

Fine-tuning with Mixed Data (FT-MIXED). To test whether updating the pretrained policy directly is preferable to ALTER’s strategy (freezing the pretrained policy and adding a coordination head), we set up a baseline that fine-tunes the pretrained policy using both multi-agent and single-agent data. Both fine-tuning baselines optimize all 17,446,215 parameters of the pretrained single-arm policy without adding modules or partner-specific inputs.

Fine-tuning with Multi-Agent Data Only (FT-MULTI). To test whether source replay is needed during adaptation, we also include a variant of FT-mixed that fine-tunes the pretrained policy using *only* the multi-agent demonstrations.

TABLE I: TwoArmPlaceWipe coordination success (% , $\uparrow$) across demonstration budgets. Bold denotes the best result within each data regime.

Multi-agent demos	Single-arm demos	Method	Checkpoint	Coordination success $\uparrow$
20	20	ALTER	500k	35.0
20	20	FS	200k	13.5
20	20	FT-MIXED	200k	33.0
20	0	FT-MULTI	200k	29.5
40	40	ALTER	300k	71.5
40	40	FS	900k	28.0
40	40	FT-MIXED	400k	59.5
40	0	FT-MULTI	300k	50.5
60	60	ALTER	500k	87.0
60	60	FS	400k	59.0
60	60	FT-MIXED	500k	63.5
60	0	FT-MULTI	200k	58.5

TABLE II: Source-task success (% , $\uparrow$) after two-arm adaptation.

Multi-arm demos	Single-arm demos			Source success $\uparrow$		
	Expert	Distilled rollouts	Method	Place-return	Wipe	Combined
0	600	0	Base policy	100.0	94.0	97.0
20	0	20	ALTER	99.0	95.0	97.0
20	0	20	FS	56.0	9.0	32.5
20	0	20	FT-MIXED	55.0	37.0	46.0
20	0	0	FT-MULTI	0.0	40.0	20.0
40	0	40	ALTER	100.0	95.0	97.5
40	0	40	FS	75.0	14.0	44.5
40	0	40	FT-MIXED	84.0	81.0	82.5
40	0	0	FT-MULTI	0.0	68.0	34.0
60	0	60	ALTER	99.0	95.0	97.0
60	0	60	FS	76.0	46.0	61.0
60	0	60	FT-MIXED	83.0	82.0	82.5
60	0	0	FT-MULTI	0.0	50.0	25.0

C. Two-Arm Low-Data Coordination

To test our method’s multi-agent data efficiency, we compare all methods under limited multi-agent data budgets.

Results. Tab. I reports coordination success across three data budgets. ALTER exceeds FS by 21.5, 43.5, and 28.0 percentage points at 20, 40, and 60 multi-agent demonstrations, respectively. FT-MIXED achieves 33.0%, 59.5%, and 63.5% at these budgets, exceeding FS but remaining below ALTER. FT-MULTI achieves 29.5%, 50.5%, and 58.5%, below FT-MIXED at every budget.

Key Takeaway. These results answer Q1: adapting a frozen single-arm policy with a coordination head outperforms learning from scratch when multi-agent data is limited.

D. Source-Task Retention

Coordination gains may come at the cost of single-agent skills. We therefore evaluate the same checkpoints as in Table I on the original place-and-return and wipe-and-return tasks of the source domain.

TABLE III: Effect of coordination-head size on coordination and source-task success (%, $\uparrow$) in TwoArmPlaceWipe.

Size	Method	Total params. (M)	Trainable fraction (%)	Coordination success $\uparrow$	Source success $\uparrow$
XS	ALTER	17.881	2.4	25.0	97.0
XS	FS	17.881	100.0	12.0	34.5
S	ALTER	19.077	8.5	27.0	97.0
S	FS	19.076	100.0	10.0	37.5
M	ALTER	20.815	16.2	33.5	97.0
M	FS	20.815	100.0	10.5	39.5
L	ALTER	24.846	29.8	35.0	97.0
L	FS	24.846	100.0	13.5	32.5

Results. Table II shows that ALTER achieves combined source success of 97.0%–97.5%, compared with 32.5%–61.0% for FS. FT-MIXED achieves 46.0%, 82.5%, and 82.5% at 20, 40, and 60 multi-agent demonstrations, respectively, also remaining below ALTER. FT-MULTI achieves 20.0%, 34.0%, and 25.0%, below all three other methods at each budget. The largest task-specific gap between ALTER and FS occurs on wipe-and-return. The unadapted base policy achieves 100.0% on place-return and 94% on the wipe task, with 100 evaluation episodes per source-task for the adapted policies and the base policy.

Key Takeaway. These results answer Q2: despite the addition of the coordination head, the adapted policy maintains high success on both original single-arm tasks.

E. Ablation Studies

Coordination-head size may affect both learning from limited demonstrations and source-task retention. We examine these effects by varying the head size while keeping the source policy frozen (Q3).

Setup. We evaluate heads of four different sizes on TwoArmPlaceWipe using 20 multi-agent and 20 single-arm demonstrations. We hold the base policy, data, observation interface, and training and evaluation protocol fixed across all sizes. For each size, we match an FS policy to the combined base-plus-head parameter count by varying only its transformer and feed-forward widths. Through this matching strategy, parameter counts are within 0.0013% of the desired base-plus-head parameter count. All other FS architecture choices remain fixed.

Results. Coordination success for ALTER increases from 25.0% with the XS head to 35.0% with the L head, while every reported head retains 97.0% combined source success. Each coordination-head variant achieves higher coordination and source success than its parameter-matched FS policy. We therefore use the L head in the main two-arm experiments.

Key Takeaway. These results address Q3: larger coordination heads achieve higher coordination success in this comparison, while all four sizes maintain high source success.

VII. HARDWARE EXPERIMENTS

Finally, we evaluate whether the simulation results transfer to hardware using the strongest baseline variants from Sec. VI.

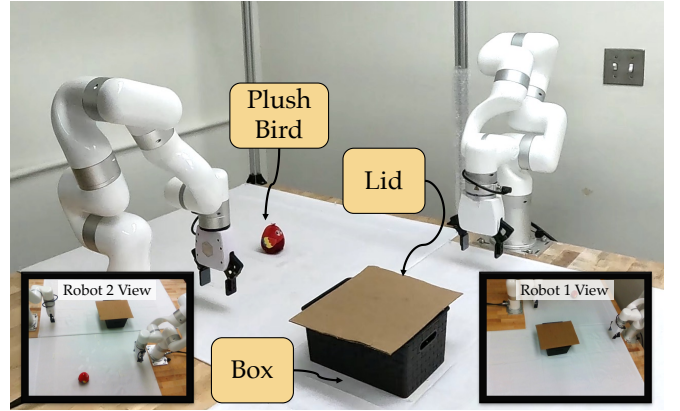


Fig. 4: Hardware setup with two xArm7 robots, a lidded box, and a plush bird. One robot removes the lid, the other places the bird in the box, and the first robot replaces the lid.

A. Setup

Task Description. In our hardware experiments, two 7-dof xArm7 robots are tasked with placing a plush bird into a lidded box. To achieve this, Robot 1 must temporarily remove the lid before Robot 2 places the bird in the open box. We consider the task successfully completed if the bird is inside the box and the lid covers more than 50% of the box while remaining in place without robot support. Fig. 4 shows the setup for this experiment.

Baselines. As baselines, we consider the strongest baselines from our simulation experiments (§VI): FT-MIXED as the strongest fine-tuning variant, and FS as a baseline that trains on all available data from scratch rather than fine-tuning.

Demonstration Data. We train each method using 69 two-arm demonstrations and 68 policy-distilled single-arm demonstrations. We pretrain the single-arm policy used by ALTER and FT-MIXED on a total of 332 single-arm demonstrations of three source tasks: lid removal, lid replacement, and bird-pick-place. Trials have a 900-timestep cutoff.

Implementation Details. Each robot receives observations from its own shoulder-view RGB camera. Both the planning and execution horizons are 20 timesteps. We train all models for 25k gradient steps. All other hyperparameters match the nominal setting of Sec. VI-A.

B. Results

Tab. IV reports the results of our hardware experiments.

Coordination. Across 20 multi-agent trials per method, we find that ALTER and FT-MIXED each succeed in 18/20 trials (90%), compared with 13/20 (65%) for FS. Our method matches the observed coordination success of full-policy fine-tuning with mixed data (FT-MIXED) and exceeds training from scratch (FS) by 25 percentage points.

Source Skill Retention. We evaluate each adapted policy on 20 trials of each source task. ALTER succeeds in all 20 trials of lid replacement and bird-pick-place, and succeeds in 19 out of 20 trials of lid removal. FS succeeds in 19/20 lid replacement trials, 12/20 bird-pick-place trials, and 9/20 lid removal trials. FT-MIXED succeeds in 11/20, 19/20, and

TABLE IV: Hardware coordination and source success.

Method	Coordination success $\uparrow$	Source success $\uparrow$		
		Lid removal	Lid replacement	Bird pick-place
Base policy	–	40/40	40/40	40/40
ALTER	18/20	19/20	20/20	20/20
FS	13/20	9/20	19/20	12/20
FT-MIXED	18/20	11/20	19/20	17/20

17/20 trials, respectively. In summary, our method ties for the highest observed coordination success while achieving the highest observed success on all three source tasks. The source-task evaluation distinguishes it from FT-MIXED, despite their equal coordination success.

VIII. CONCLUSION

We study adaptation of pretrained single-arm base policies to coordinate on demand, i.e., to achieve decentralized multi-agent coordination while retaining the ability to also act independently. We study this problem under two key constraints: limited access to multi-agent coordination demonstrations and no access to the expert demonstrations underpinning the pretrained single-arm policy. Our method, ALTER, freezes the pretrained policy and trains a residual coordination head using both multi-agent demonstrations and policy-distilled single-arm replay data. In simulation, ALTER improves coordination success over parameter-matched training from scratch by up to 43.5 percentage points and over full-policy fine-tuning with mixed data by up to 23.5 percentage points. It also improves combined source-task success by up to 64.5 and 51.0 percentage points over those baselines, respectively. Hardware experiments on a collaborative manipulation task between two 7-dof xArm7 robots support these findings. Future work will extend the approach to more diverse tasks and robot platforms.

REFERENCES

- [1] Z. Sun, Y. Peng, Y. Meng, X. Li, Y. Sun, H. Jiang, B.-S. Huang, Z. Bing, X. Wang, and A. Knoll, “Robotdancing: Residual-action reinforcement learning enables robust long-horizon humanoid motion tracking,” *IEEE Robotics and Automation Letters*, 2026.
- [2] J. Han, W. Xie, J. Zheng, J. Shi, W. Zhang, T. Xiao, and C. Bai, “Kungfubot2: Learning versatile motion skills for humanoid whole-body control,” *arXiv preprint arXiv:2509.16638*, 2025.
- [3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [4] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [5] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [6] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “RDT-1B: A diffusion foundation model for bimanual manipulation,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [7] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [8] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” *arXiv preprint arXiv:2205.09991*, 2022.
- [9] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” *arXiv preprint arXiv:2403.03954*, 2024.
- [10] D. Dong, M. Bhatt, S. Choi, and N. Mehr, “Mimic-d: Multi-modal imitation for multi-agent coordination with decentralized diffusion policies,” *arXiv preprint arXiv:2509.14159*, 2025.
- [11] R. Doshi, T. Gao, A. Chen, C. Finn, and J. Bohg, “Chorus: Decentralized multi-embodiment collaboration with one vla policy,” *arXiv preprint arXiv:2606.12352*, 2026.
- [12] Z. Zhu, M. Liu, L. Mao, B. Kang, M. Xu, Y. Yu, S. Ermon, and W. Zhang, “Madiff: Offline multi-agent learning with diffusion models,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 4177–4206, 2024.
- [13] C. He, G. S. Camps, X. Liu, M. Schwager, and G. Sartoretti, “Latent theory of mind: A decentralized diffusion architecture for cooperative manipulation,” *arXiv preprint arXiv:2505.09144*, 2025.
- [14] L. Peters, L. Ferranti, A. Bajcsy, and J. Alonso-Mora, “Coordinated diffusion: Generating multi-agent behavior without multi-agent demonstrations,” *arXiv preprint arXiv:2605.11485*, 2026.
- [15] S. Auddy, J. Hollenstein, M. Saveriano, A. Rodriguez-Sanchez, and J. Piater, “Continual learning from demonstration of robotics skills,” *Robotics and Autonomous Systems*, vol. 165, p. 104427, 2023.
- [16] T. Liu, J. Li, Y. Zheng, H. Niu, Y. Lan, X. Xu, and X. Zhan, “Skill expansion and composition in parameter space,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 85 192–85 228.
- [17] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [18] S. Huang, Z. Zhang, T. Liang, Y. Xu, Z. Kou, C. Lu, G. Xu, Z. Xue, and H. Xu, “Mentor: Mixture-of-experts network with task-oriented perturbation for visual reinforcement learning,” *arXiv preprint arXiv:2410.14972*, 2024.
- [19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [20] J. O. Zhang, A. Sax, A. Zamir, L. Guibas, and J. Malik, “Side-tuning: A baseline for network adaptation via additive side networks,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [21] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [22] T. Karras, M. Aittala, T. Aila, and S. Laine, “Elucidating the design space of diffusion-based generative models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [23] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [24] B. Efron, “Tweedie’s formula and selection bias,” *Journal of the American Statistical Association*, vol. 106, no. 496, pp. 1602–1614, 2011.
- [25] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- [26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [27] A. Robins, “Catastrophic forgetting, rehearsal and pseudorehearsal,” *Connection Science*, vol. 7, no. 2, pp. 123–146, 1995.
- [28] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [29] H. Shin, J. K. Lee, J. Kim, and J. Kim, “Continual learning with deep generative replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [30] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.